
大数据与人工智能
自动化学院

自然语言处理之

《ATTENTION IS ALL YOU NEED》

1. 引言

- 《Attention is all you need》论文是2017年谷歌团队发表的一篇论文，
- 这是一篇非常经典的**自然语言处理领域**的文章，基于这篇文章，才有了近几年非常火的bert， Albert, LLAMA, ChatGPT等模型

- 这篇文章最大的亮点就是提出了一种Transformer的结构
- 这种结构是完全依赖注意力机制来刻画输入和输出之间的全局依赖关系，而不使用递归运算的RNN网络了
- 这样的好处就是:
  - 第一可以有效的防止RNN存在的梯度消失的问题，
  - 第二是允许所有的字全部同时训练（RNN的训练是迭代的，一个接一个的来，当前这个字过完，才可以进下一个字），即并行训练，大大加快了计算效率。

- Transformer使用了位置嵌入来理解语言的顺序
- 使用了多头注意力机制**和全连接层等进行计算
- LayerNorm机制
- Encoder-Decoder架构
- Multi-Head Attention机制

论文：https://arxiv.org/abs/1706.03762

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

大纲如下：

- Abstract
- Introduction
- Model Architecture（这一部分占据很大一部分，详细剖析）
- Training
- Conclusion

2. ABSTRACT

- 目前用于序列转换的模型依然是Encoder-Decoder结构的RNN或者CNN。
- 效果比较好的是Encoder-Attention-Decoder这样的结构。

所以在这里作者基于Encoder-Decoder提出了一种完全依赖Attention机制的Transformer模型

- 可以并行化而减少训练时间，
- 实验表明，效果很好

3. INTRODUCTION

- 以往RNN结构的弊端：
  - 需要递归迭代运行，没法并行化，
  - 这样对于很长的句子来说，很可能出现梯度消失的情况，
  - 计算量也很大，速度比较慢。

所以需要改进。

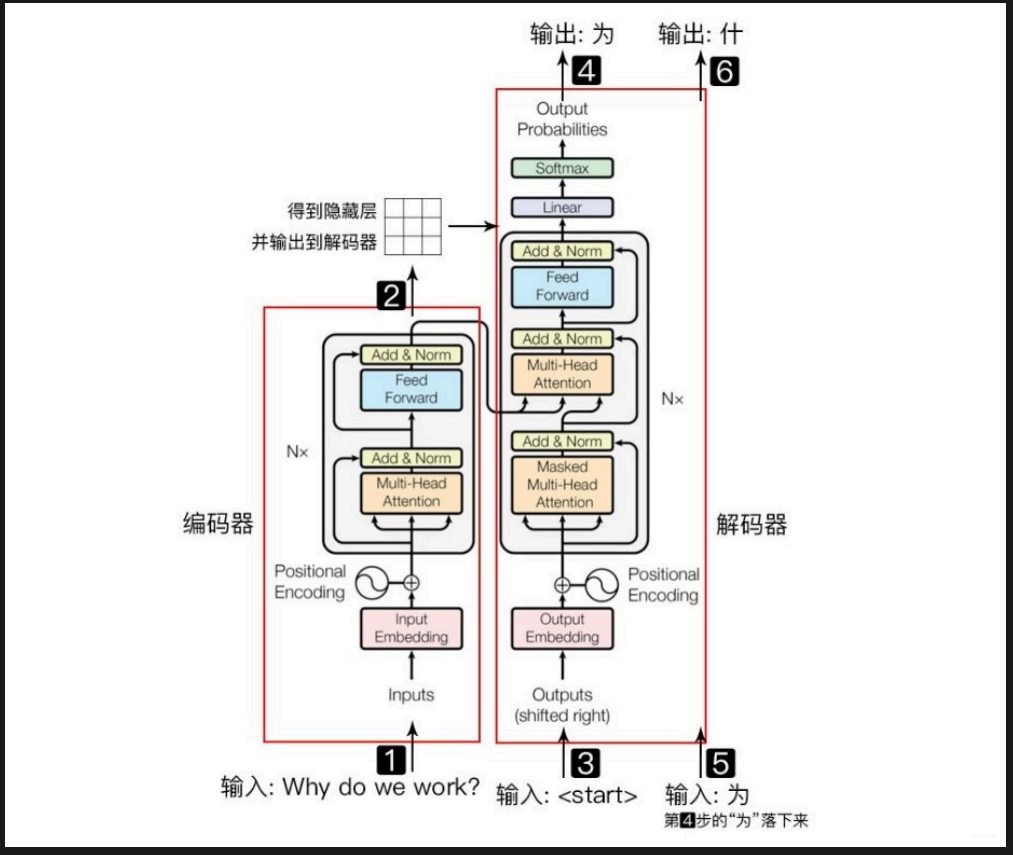
- Attention是利用局部聚焦的思想去建立注意力模型
- 但目前这样的机制都是和RNN连接
- 本文提出的self-attention自注意：将单个序列的不同位置联系起来以计算序列表示的注意机制

本文提出了一种Transformer模型

这种模型不用RNN或者说CNN这种递归机制，而是完全依赖于Attention。

4. MODEL ARCHITECTURE（重点）

先看总体结构：



- 从宏观角度上看结构
  - 可以看到Transformer模型用了Encoder-Decoder结构
  - 编码器部分负责把自然语言序列映射成为隐藏层（就上面那个九宫格）
  - 含有自然语言序列的数学表达，
  - 然后解码器把隐藏层再映射（预测）为自然语言序列，

可以解决各种问题，比如情感分类，命名实体识别，语义关系抽取，机器翻译，摘要生成等等。

上面的结构的工作流程

比如做一个机器翻译

- Why do we work? -> 为什么要工作?

下面是操作流程:

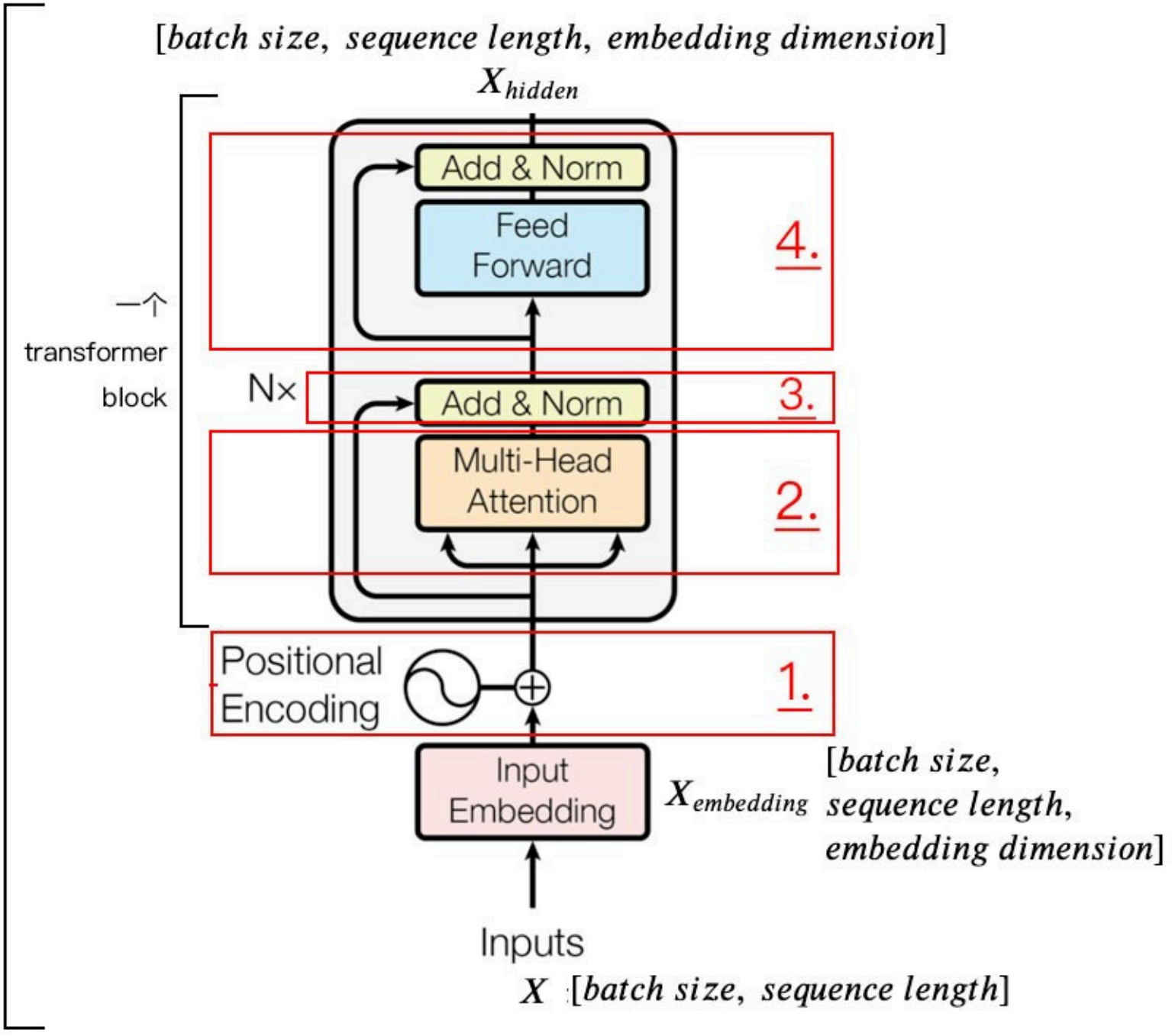
1. 输入自然语言序列： Why do we work?
2. 编码器输出的隐藏层是Why do we work的一种数学表示，类似于提取了每一个词的信息，然后汇总，转换成了这句话的数学向量。然后输入到解码器
3. 输入符号到解码器
4. 就会得到第一个字“为”

5. 将得到的第一个字“为”落下来再输入到解码器
6. 得到第二个字“什”
7. 将得到的第二个字落下来输入，得到“么”，重复，直到解码器全部输出。
8. 翻译完成

下面重点讲讲细节部分

看看究竟是怎么得到数学向量的，以及怎么通过数学向量得出最终答案的?

从自然语言序列
经过计算
到隐藏层的过程



看结构：

- 编码器部分是由 Nx个transformer block堆叠而成的
- 拿一个transformer block来进一步观察，每一个transformer block又有两个子层
  - 第一个是Multi-Head Attention多头注意力部分
  - 第二个是feed-forward部分。

我们输入句子：Why do we work? 的时候，它的编码流程进一步细化：

1. 首先输入进来之后，经过Input Embedding层，对每个字进行embedding编码（这个后面会说），然后再编入位置信息（position Encoding），形成带有位置信息的embedding编码。
2. 然后进入多头注意力部分，这部分是多角度的self-attention部分，在里面每个字的信息会依据权重进行交换融合，这样每一个字会带上其他字的信息（信息多少依据权重决定），
3. 然后进入feed-forward部分进行进一步的计算
4. 最后就会得到输入句子的数学表示了。

下面再详细说一下每一部分的细节：

3.1.1 位置嵌入

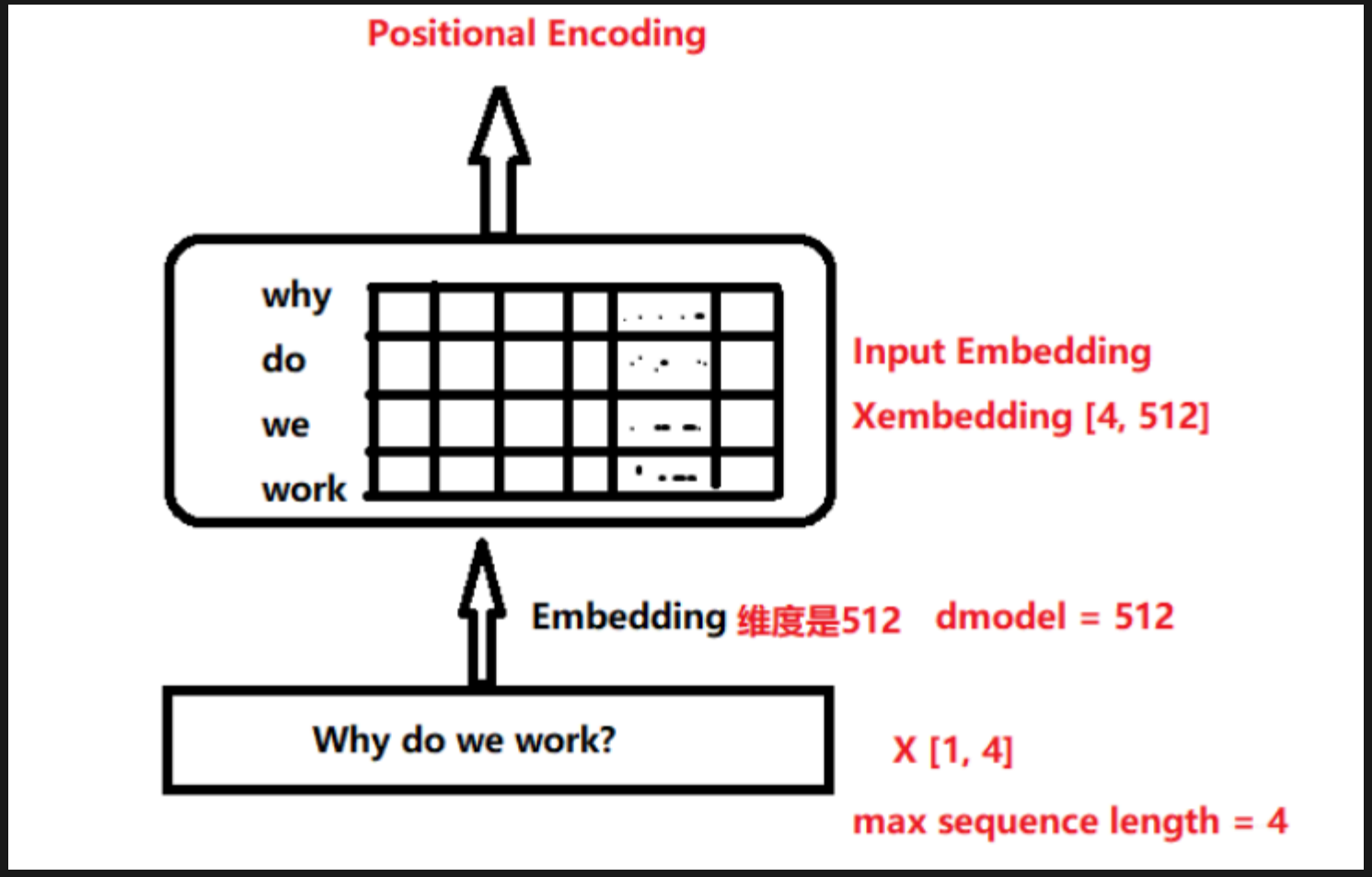
必须提供每个字的位置信息给transformer, 才能识别出语言中的顺序关系。

- 定义一个位置嵌入的概念，也就是$positional\ encoding$,
- 位置嵌入的维度为[$max\ sequence\ length$, $embedding\ dimension$], 嵌入的最大长度&词向量的维度
- $max\ sequence\ length$属于超参数, 指的是限定的最大单个句长

注意：

- 一般以word(字)为单位训练transformer模型
- 首先要初始化字向量为$[vocab\ size, embedding\ dimension]$
  - $vocab\ size$为总共的字数
  - $embedding\ dimension$为字向量的维度, 也是每个字的数学表达

举例子来看：


Positional Encoding
why
do
we
work
Input Embedding
Xembedding [4, 512]
Embedding 维度是512
dmodel = 512
Why do we work?
X [1, 4]
max sequence length = 4

这里论文里面使用了sine和cosine函数的线性变换来提供给模型字的位置信息：

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

- 上式中pos指的是句中字的位置, 取值范围是

$$[0,\ \text{max sequence length})$$

- i指的是编码向量的维度, 取值范围是

$$[0,\ \text{embedding dimension})$$

- d_{model}是编码向量的维度最大值

实现编码的示例代码如下：

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

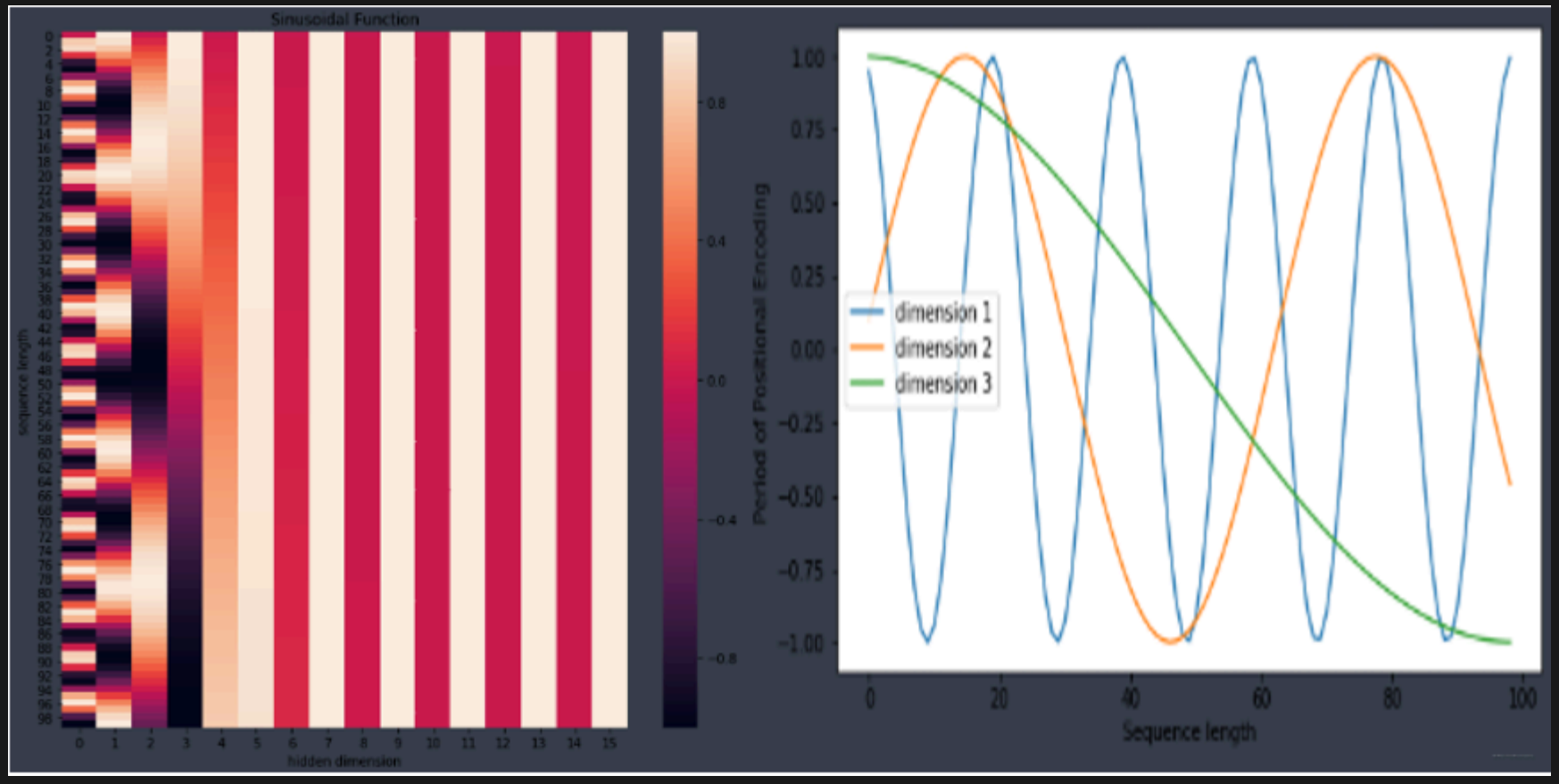
def get_positional_encoding(max_seq_len, embed_dim):
    """
    生成位置编码矩阵

    参数:
    max_seq_len (int): 句子的最大长度，决定了位置编码矩阵的行数
    embed_dim (int): 词嵌入的维度，决定了位置编码矩阵的列数
    """

    # 初始化一个空列表，用来存放最终的矩阵
    all_rows = []
```

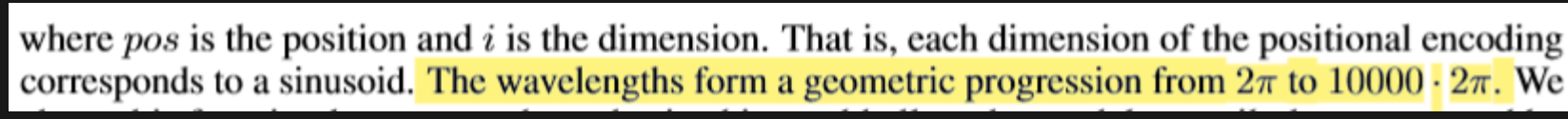
上面例程中：字的位置 pos = 0, ..., 99（最长100个字的输入） 编码向量 i = 0, ..., 15（16位编码）

把位置信息编码可视化一下，最后得到这样的效果：

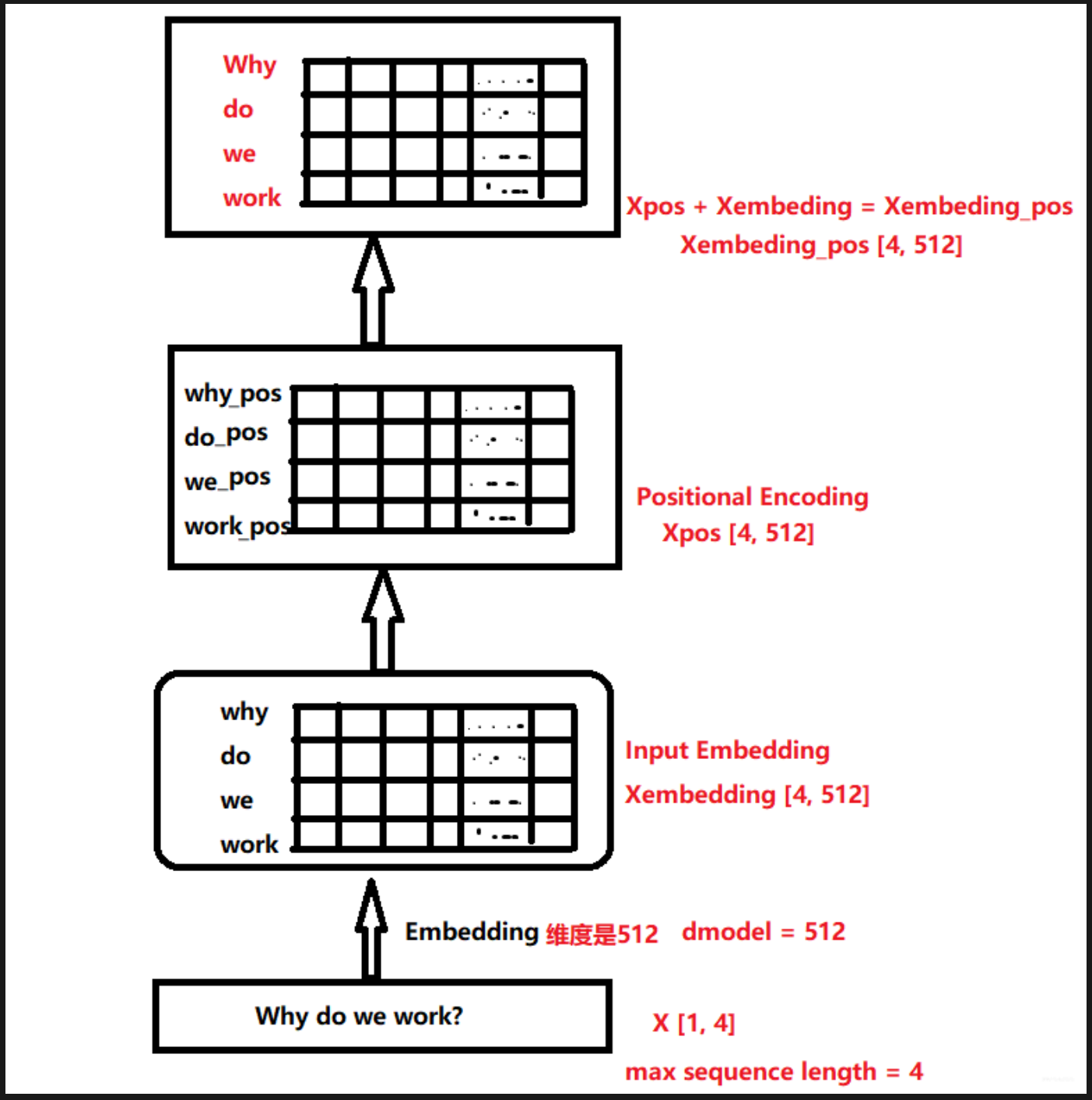


- 所以， 会得到Why do we work这四个词的位置信息，
- 然后Embedding矩阵和pos位置矩阵的加和作为带有位置信息的新X
- 新X的字编码称为Xembedding_pos

- sin和 cos一组公式, 也就是对应着 embedding dimension编码向量i维度的一组奇数和偶数的序号的维度,
- 例如 0,1一组, 2,3一组, 分别用上面的 sin和 cos函数做处理, 从而产生不同的周期性变化,
- 在 embedding dimension编码向量维度上随着维度序号i增大, 字的位置pos的周期变化会越来越慢, 而产生一种包含位置信息的纹理, 就像论文原文中第六页讲的


where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We

- 位置嵌入函数的周期从 2π（i=0）到 10000*2π（i=100）变化
- 而每一个位置在embedding dimension维度上都会得到不同周期的 sin和 cos函数的取值组合
- 从而产生独一的纹理位置信息, 模型从而学到位置之间的依赖关系和自然语言的时序特性.


Why
do
we
work
Xpos + Xembeding = Xembeding_pos
Xembeding_pos [4, 512]
why_pos
do_pos
we_pos
work_pos
Positional Encoding
Xpos [4, 512]
why
do
we
work
Input Embedding
Xembedding [4, 512]
Embedding 维度是512 dmodel = 512
Why do we work?
X [1, 4]
max sequence length = 4

这里再补充两个问题

- 第一个就是为什么要用这种方式编码呢?

好处：

1. **独特性：** 每个位置（pos=1, pos=2...）都会得到一个独一无二的"波纹"向量。
2. **相对位置：** 这种数学设计有一个巧妙之处：模型能非常容易地学会"相对位置"。比如，它能学会"任意位置+2"和"任意位置"之间的关系。这让模型能理解"A后面是B"这种概念，而不管A在句子的哪个位置。

- 第二个问题：这里为啥这里的单词embedding和位置embedding能直接相加呢?
- 论文里面提到原因：维度相同，因此可以相加
  - 这种加法结合了词语的语义信息（来自单词嵌入）
  - 还结合了位置信息（来自位置嵌入）
  - 使得每个词在输入到模型时都携带了完整的上下文信息。

提问：Transformer模型中位置编码的作用是什么？

- A. 增强模型的非线性能力
- B. 提供词语之间的顺序信息
- C. 增加模型的参数数量
- D. 提高训练速度

参考答案：B

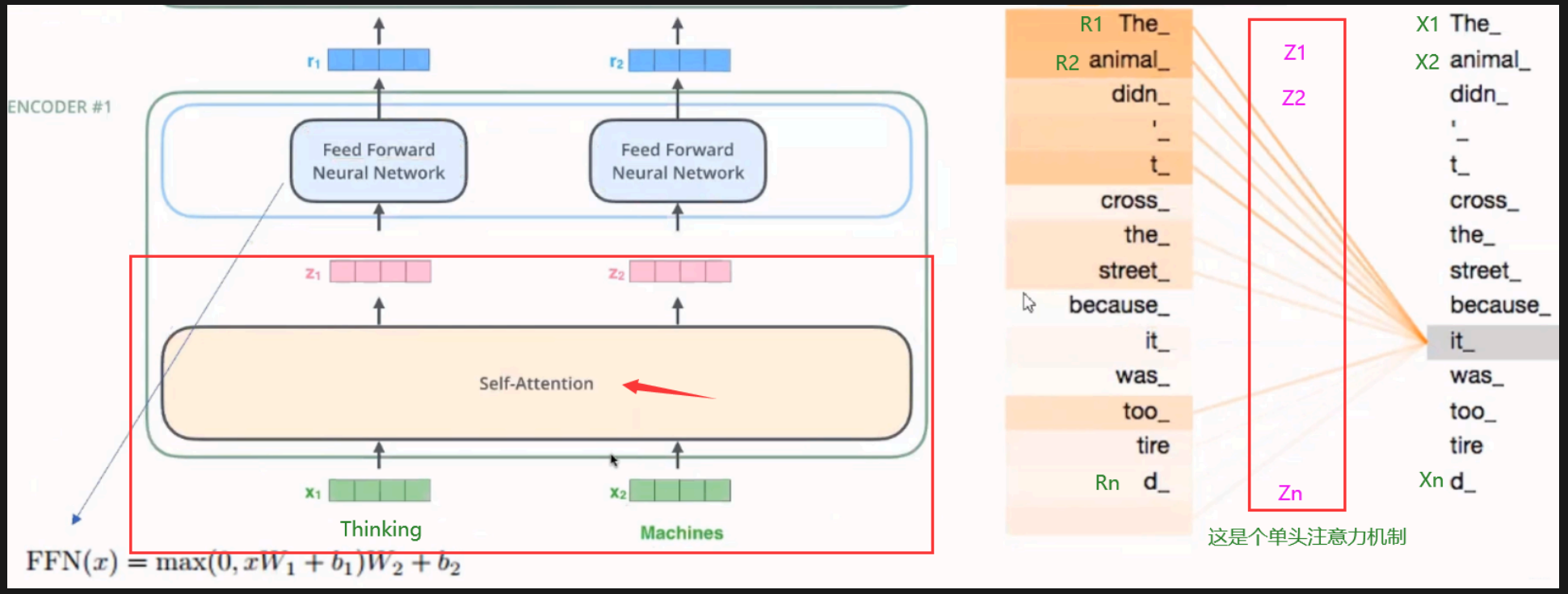
选项分析：

- A: 位置编码与非线性能力无关。
- B: 正确答案，位置编码用于提供词语之间的顺序信息。
- C: 位置编码不直接增加参数数量。
- D: 位置编码不直接提高训练速度。

3.1.2 多头注意力机制

- 这一步为了学到多重语意含义的表达，进行多头注意力机制的运算
- 多头注意力：其实就是用到了几个矩阵运算
- 先不用管怎么运算的，我们先宏观看一下这个注意力机制到底在做什么？

拿单头注意力机制举例：



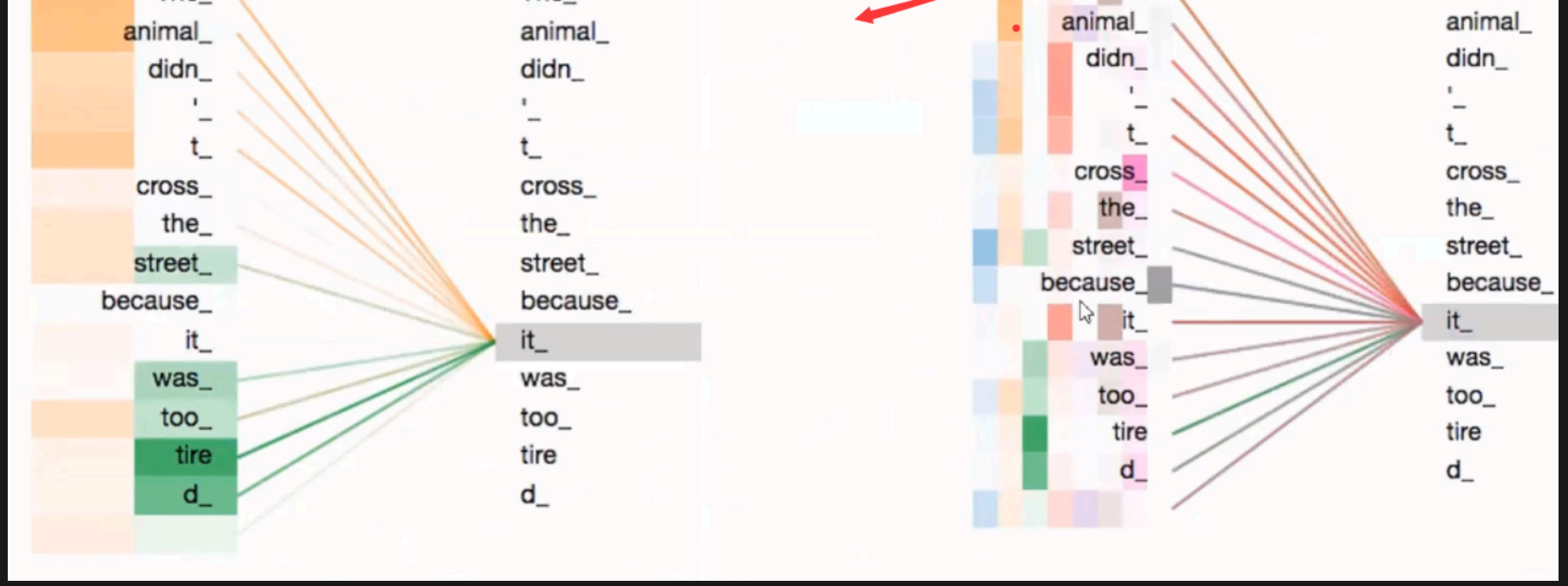
- 左边的红框就是我们现在要讲的部分，
- 右图就是单头注意力机制做的事情

拿句子举例：

```
The animal didn’t cross the street, because it was too tired.
```

- 我们看右图中：it这个词最后得到的R矩阵里面，就会表示出这个 it到底是指的什么，
- 通过橙色注意力（单头）可以看到 R1和 R2和 it最相关，就可以认为 it表示的是The animal。

- 也就是说，每一个句子经过Self-Attention映射之后都会对应一个R矩阵
- 这个橙色注意力对应的R矩阵就是表示这个字与其他字之间某个角度上的关联性信息
- 这叫做单头注意力机制（具体怎么做到的，下面会说）



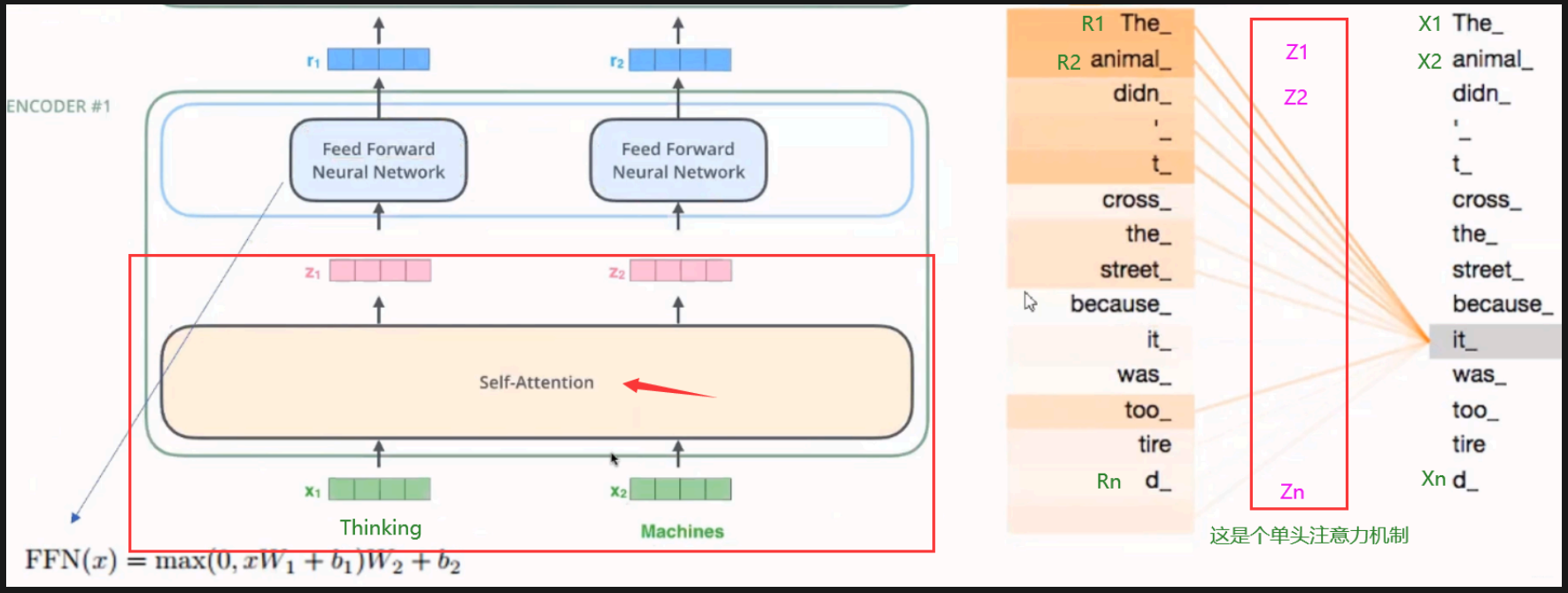
- 左边这个是两头的注意力机制，上面说到这个橙色注意力反映了 it这个词**指代的信息**。
- 而这个绿色的这个注意力，反应了it这个词的**状态信息**，
- 可以看到 it经过这个绿色注意力机制后，tired这个词与 it关联最大，

- 就是说it，绿色注意力映射之后，会更关注 tired 这个词，因为这个正好是它的一个状态。 它累了。

多头注意力的意义：

- **每个字经过多头注意力机制(比如N个头)之后会得到一个R矩阵(有一个维度是N)**
- 这个R矩阵表示这个字与其他字**在N个角度上（比如指代，状态...）的一个关联信息**
- 这N个角度就是用多头(一共N个头)的注意力矩阵体现的

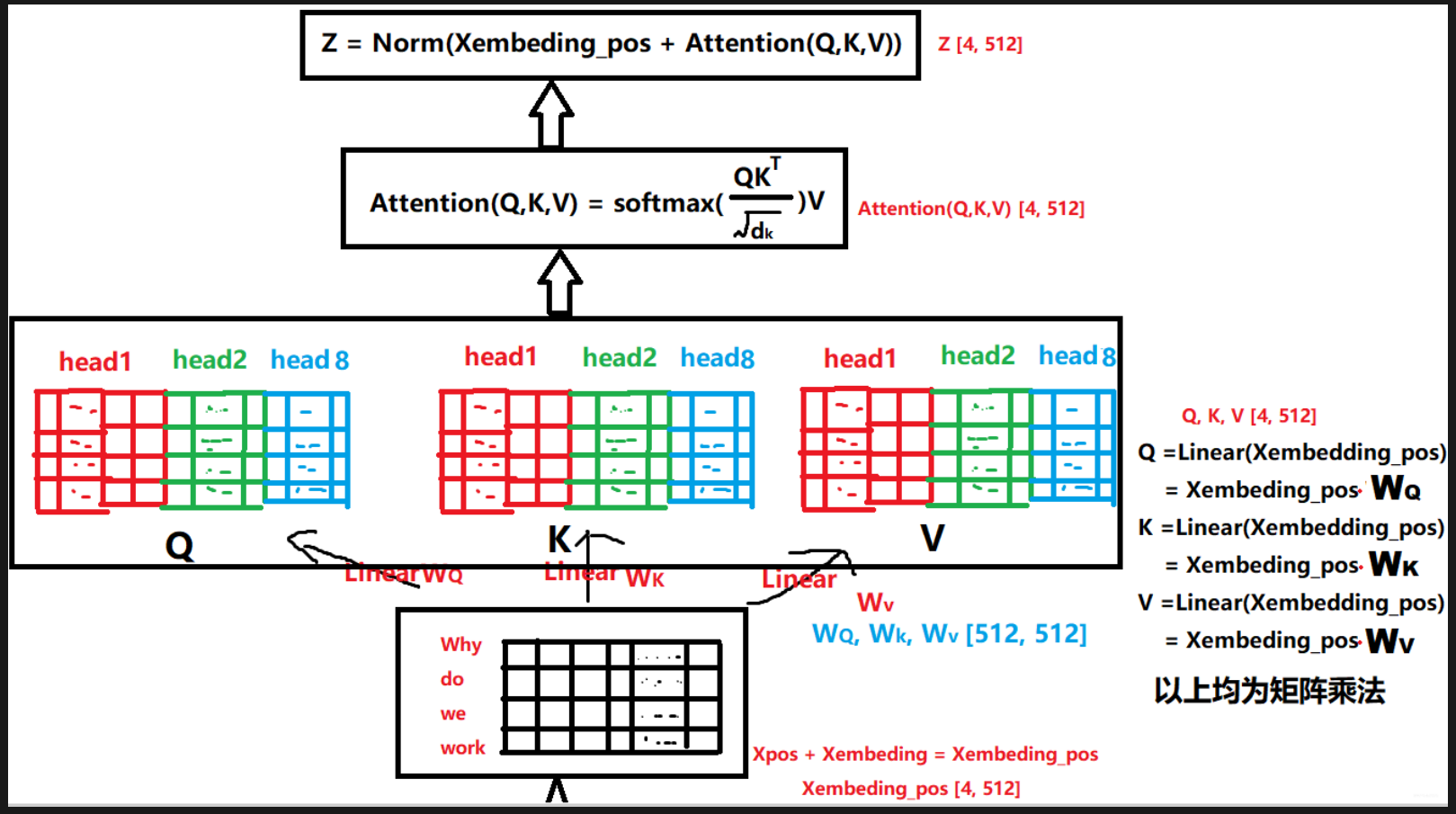
- 究竟是怎么实现的呢？ 其实这个过程中就是借助了三个矩阵X, Z, R来完成的。



- 下面具体看一下：
  - 目标是把我们的输入Xembedding_pos通过多头注意力机制（系列线性变换）先得到Z。
  - 然后Z通过前馈神经网络得到R。
  - 这个R矩阵表示这个字与其他字在N个角度上（比如指代，状态...）的一个关联信息。

先看看怎么得到这个Z:

- 在Xembedding_pos->Z的过程中到底发生了什么呢？ ``



- 首先Xembedding_pos会做三次线性（乘线性变换矩阵 W_q, W_k, W_v）变化得到Q,K,V，三个矩阵
    - 其中的线性变换矩阵 W_q, W_k, W_v，这些矩阵是可训练的参数，用于将输入嵌入向量转化为查询、键和值。
    - **生成方式**：
        - 初始时，这些矩阵是随机初始化的。
        - 在训练过程中，通过反向传播不断更新，以优化模型性能。
- 然后里面Attention机制，把Q,K,V三个矩阵进行运算，

- 最后把Attention矩阵和Xembedding_pos加起来就是Z。

为什么要这么做呢?

Q,K,V又分别表示什么意思呢?

- **Query (Q) - 查询向量：** 代表"我 (这个词) 准备去查找信息"。
- **Key (K) - 键向量：** 代表"我 (这个词) 包含什么信息，供他人查找"。
- **Value (V) - 值向量：** 代表"我 (这个词) 真正的含义是什么"。

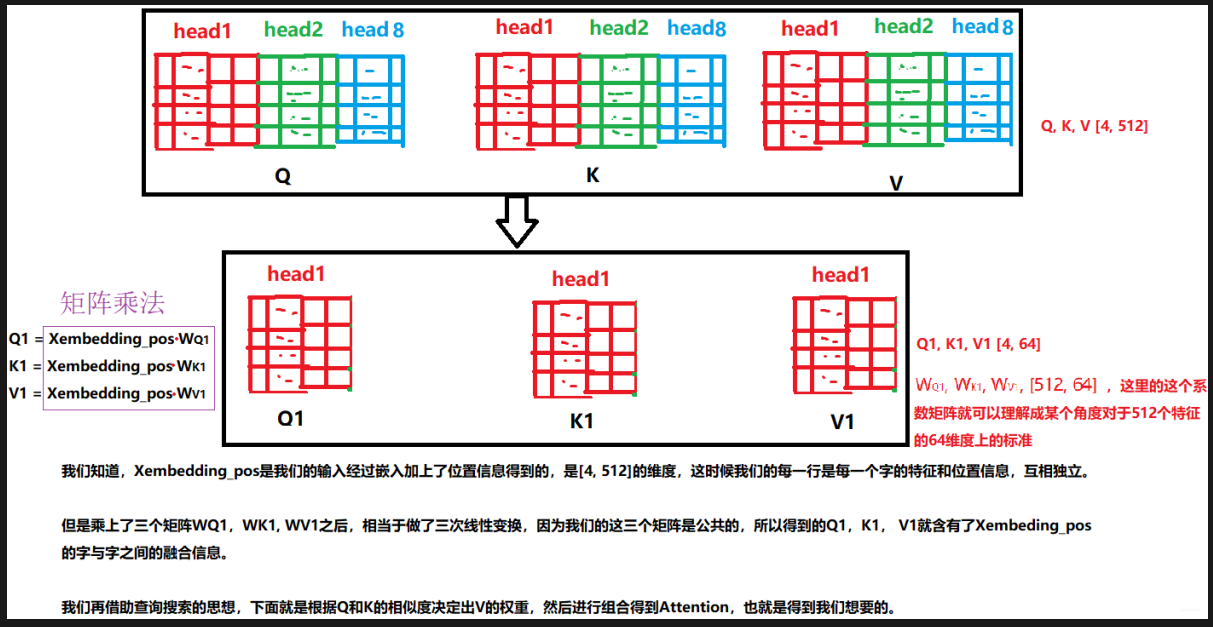
- **工作流程 (以"it"为例):**
    1. "it"的 **Q** 向量 (查询) 去和句子中_所有词_的 **K** 向量 (键) 进行"匹配" （做点积）。
    2. "it"的Q vs "animal"的K -> 匹配度90分
    3. "it"的Q vs "street"的K -> 匹配度10分
    4. "it"的Q vs "tired"的K -> 匹配度60分

- **工作流程 (以"it"为例)：**
    5. 这些"匹配度分数"（90, 10, 60...）就是"注意力权重"。
    6. 最后，用这些"权重"去**加权求和**所有词的 V 向量 (真正含义)。
    7. **结果：** "it"的新向量 = 90%的"animal"含义 + 10%的"street"含义 + 60%的"tired"含义...
    8. 这样一来，"it"的新向量就"吸收"了它最相关的词的含义，变得更懂上下文了。
- **"自注意力" (Self-Attention)：** 因为Q, K, V都来自_同一句话_，所以叫"自"注意力（句子在看自

ㄹ)。

下面具体来看一下：

- 上面图中有8个head， 我们这里拿一个head来看一下做了什么事情：
- 请注意这里head的个数一定要能够被embedding dimension整除才可以，
- 上面的embedding dimension是512， head个数是8，那么每一个head的维度是(4, 512/8)，即(4, 64)



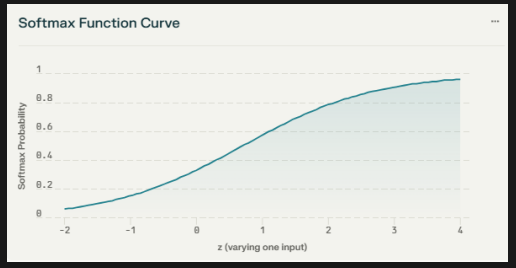
- 所有头部并行处理各自的 Qi、Ki 和 Vi 矢量（i表示是第i个头部）。
- 所有头部的输出被拼接起来形成Q, K, V
- **多样化焦点**：每个头部$head_i$可以学习数据中的不同关系和依赖关系，例如关注句子的不同部分或捕获不同类型的句法或语义信息。

- 自注意力的巧妙之处就在于这里
- 每个词向量两两之间内积，就能得到当前词与其他词的相似关系
- 有了相似关系，再通过softmax映射出权重
- 再把这个权重，再次乘到各自词语V的embedding身上，
- 再加权求和，就相当于融于了其他词的相关信息。

SOFTMAX函数

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

- 其中，$\mathbf{z}$ 是一个包含多个分数的向量。
- e^{z_i} 是对每个分数的指数运算。
- 分母是所有分数指数运算结果的总和，确保输出为概率分布（总和为1）。



- **输入变化**：横轴上显示的是单个输入值zi的变化范围。在图中，它从-2到4。其他i的输入值zi，可以固定为0.
- **连续性**：可以将其中一个输入值zi视为连续变量，从而绘制出其对输出概率的影响。

$$\text{Attention}(Q,K,V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Attention(Q,K,V) [4, 512]

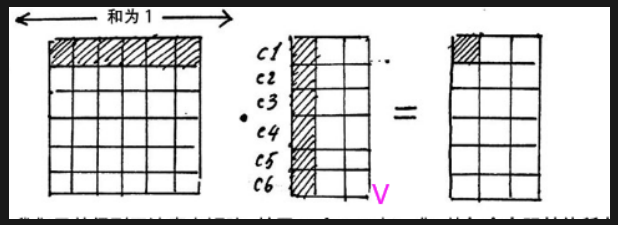
这里再解释下为啥会 QK^T 然后除以 $\sqrt{d_k}$ 的操作

- 作者指出当dk很大的时候， 这里的内积结果也会变大

[4]To illustrate why the dot products get large, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, has mean 0 and variance d_k.

- 也就是：内积的向量是均值0，方差dk（变大）
- 所以 QK^T 然后除以 $\sqrt{d_k}$ 的操作，让标准差变回为1

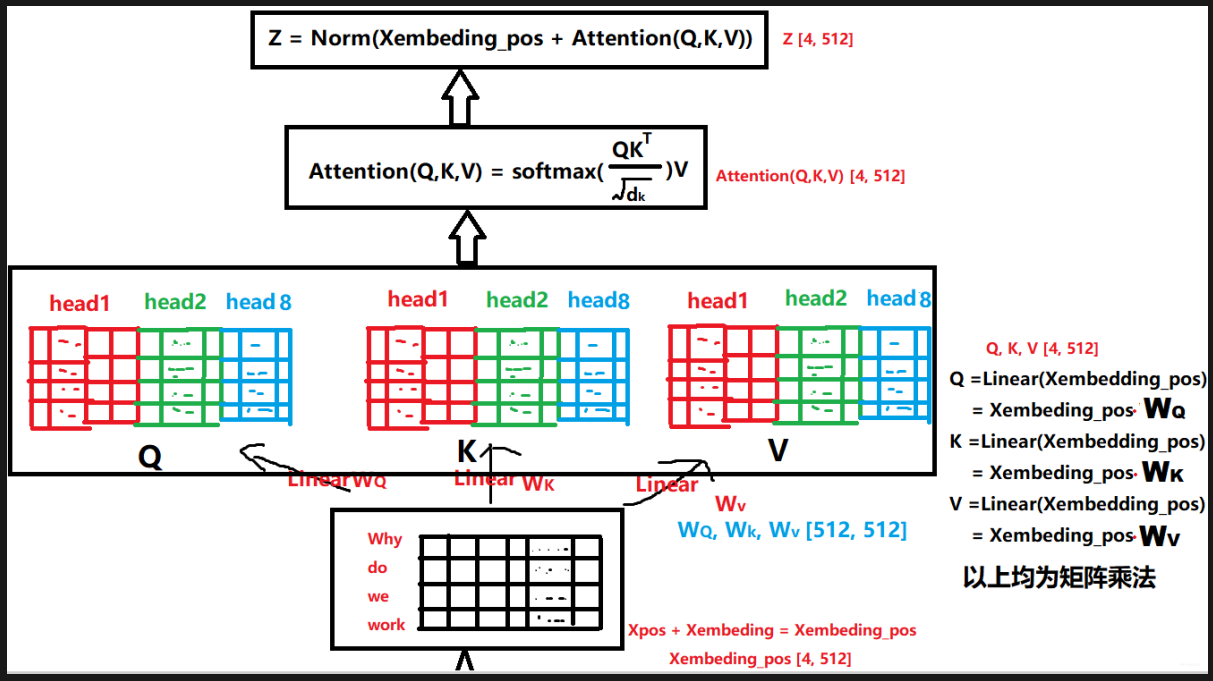
- **对每一行使用softmax归一化变成某个字与其他字的注意力的概率分布**（使每一个字跟其他所有字的权重的和为1）。



- 这时候，我们从注意力矩阵取出一行（和为1），
- 然后依次**点乘V**的列，因为矩阵V的每一行代表着每一个字向量的数学表达，

- 这样操作，得到的正是**每个字加权了注意力权重的新的字X_attention**

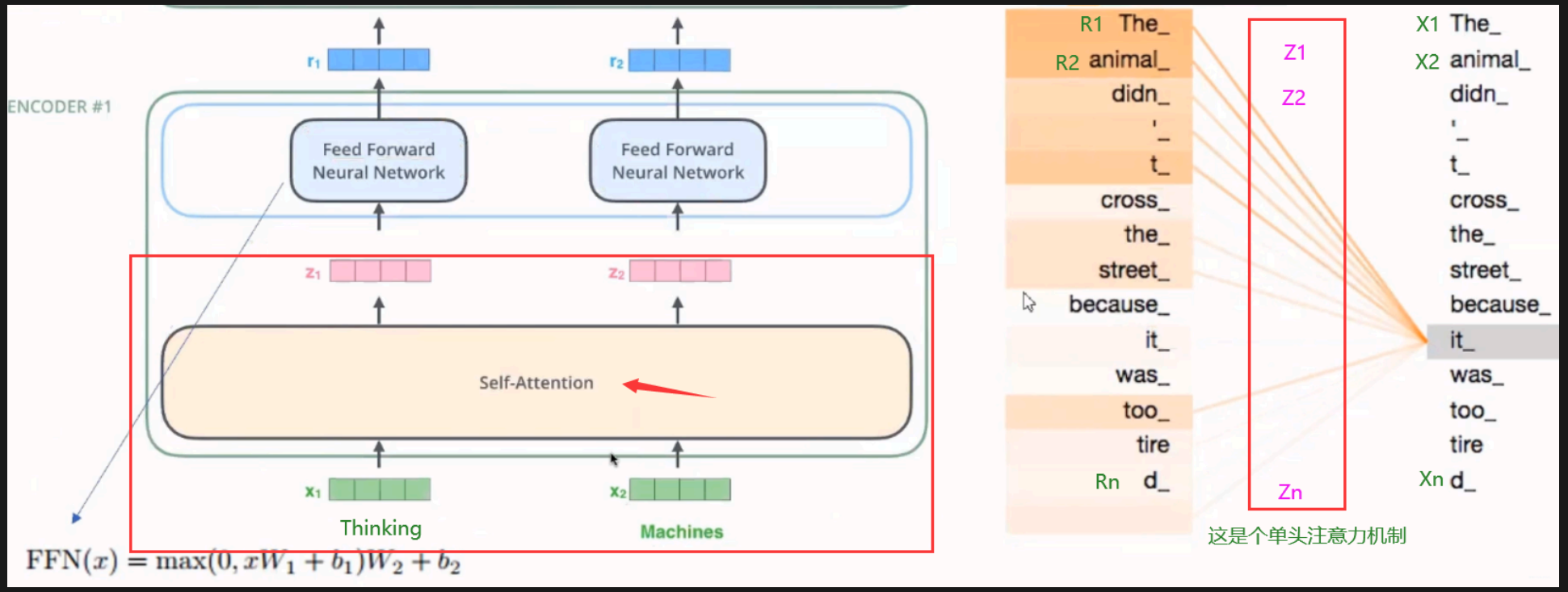
- 从而使每个字向量都含有当前句子的所有字向量的信息。
- 新的X_attention(这个X_attention中每一个字都含有其他字的信息)。



- 用这个加上之前的Xembedding_pos得到**残差连接**
- 训练的时候可以**使得梯度直接走捷径反传到最初层，不易消失**
- 另外，这里用残差还有个好处就是能够保留原始的一些信息

LAYERNORMLIZATION操作得到Z

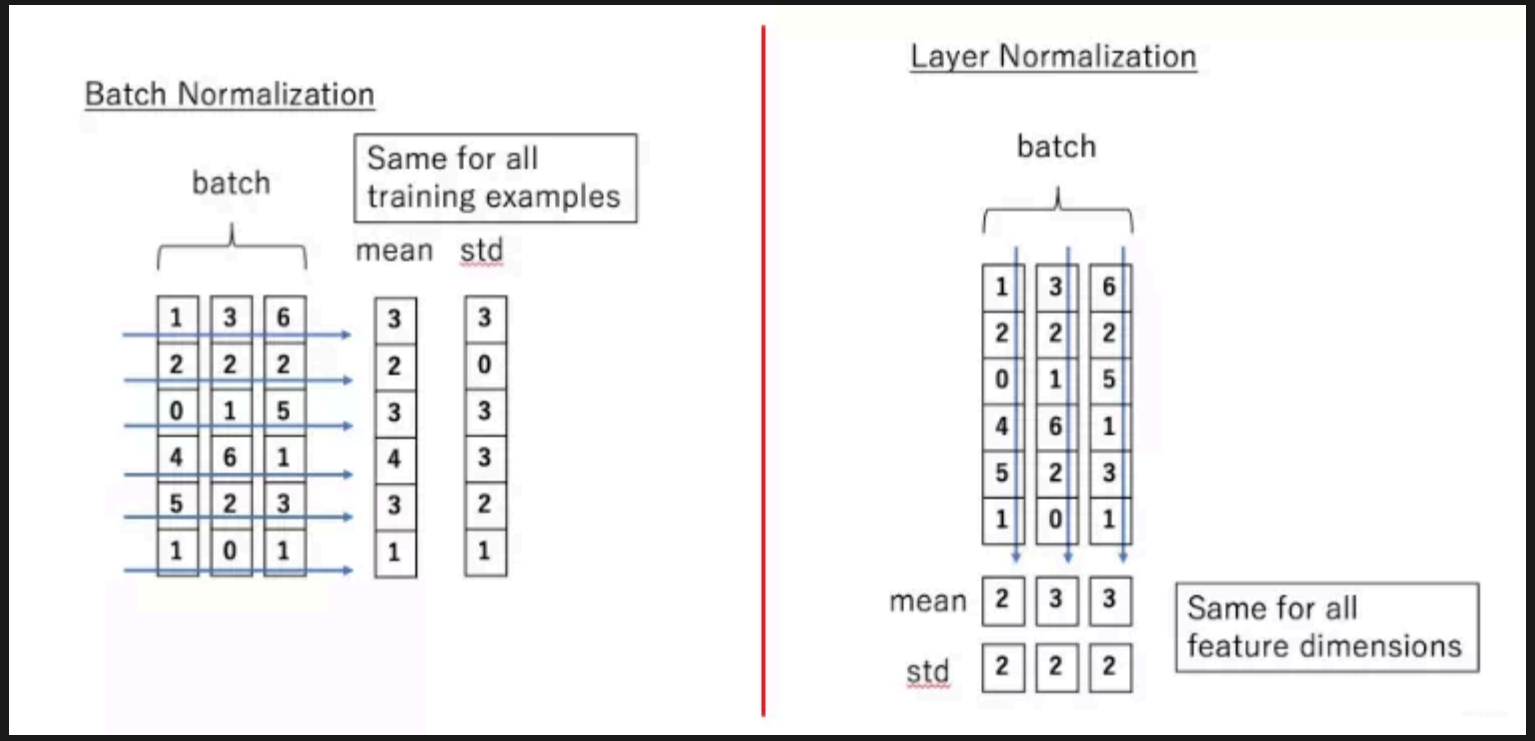
回顾前面图片



- 再经过一个LayerNormlization操作就可以得到Z。

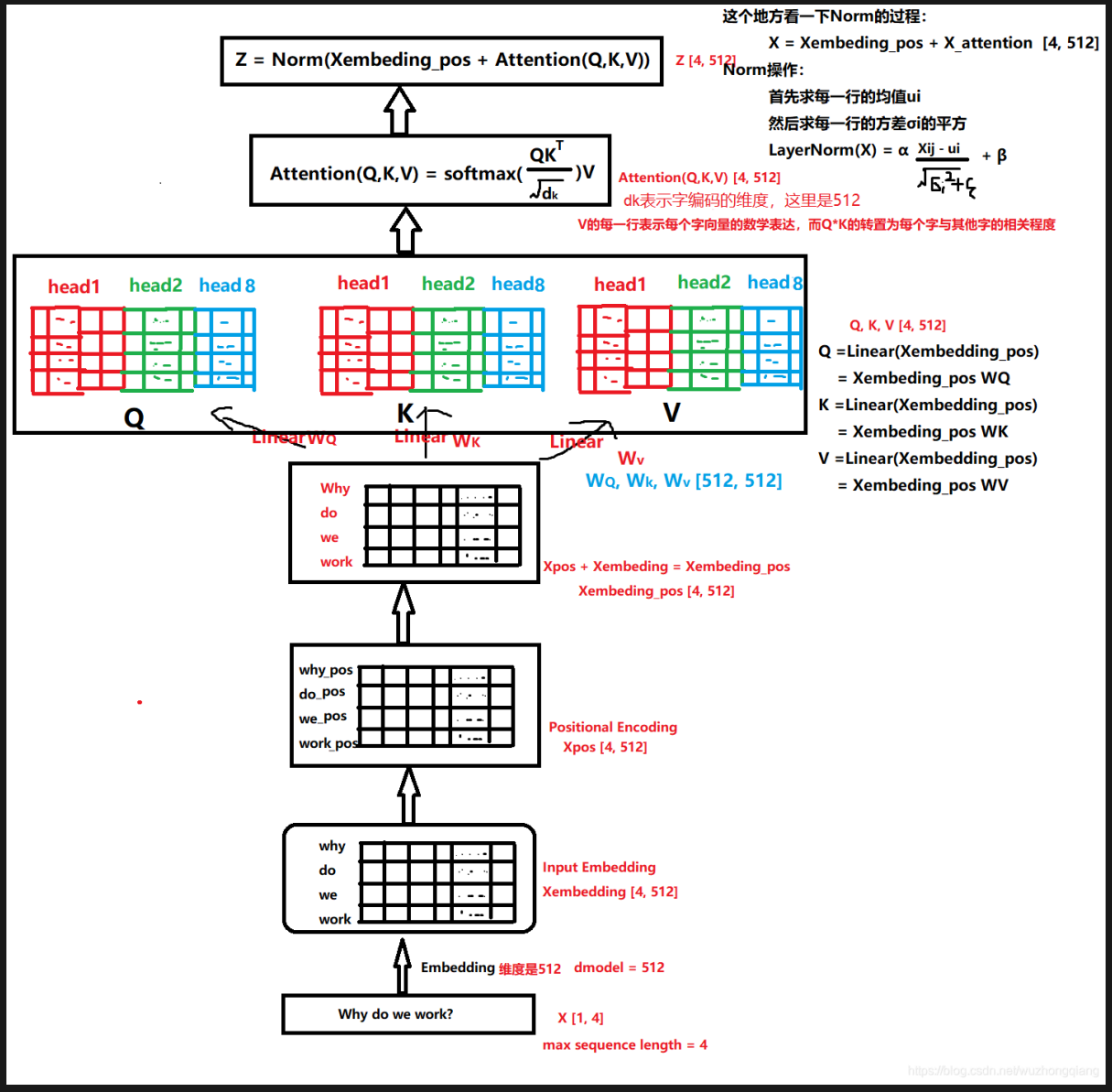
- LayerNormlization的作用是把神经网络中隐藏层归一化为标准正态分布，起到加快训练速度，加速收敛的作用。
- 类似于BatchNormlization，但是LayerNormlization不同
  - 前者是以特征为单位（每一行特征减去每一行特征的均值然后除以每一行特征的标准差），
  - 后者是一个网络层为单位（每一个网络层减去它的均值然后除以它的标准差）。

来个图感受下就是这样：



- 每个句子（样本）毕竟差距还是蛮大的，
- 最好还是各个句子归一化自己的——即Layer Normalization

- 多头注意力机制细节总结起来就是下面这个图



了：

3.1.3 前馈神经网络（FEEDFORWARD）

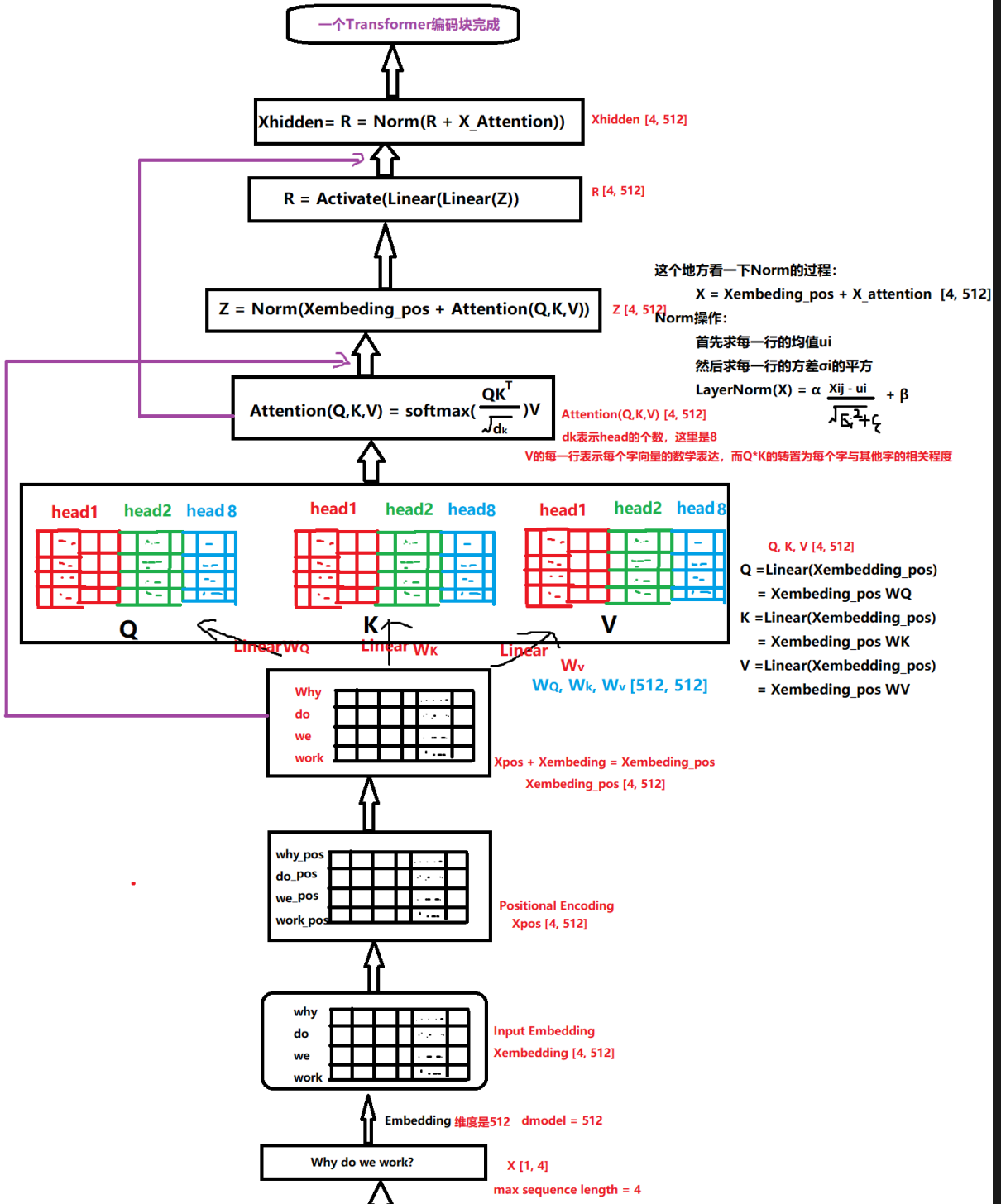
- 这一块比较简单，我们上面通过多头注意力机制得到了Z，
- 下面就是把Z再做两层线性变换，

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- 然后relu激活就得到最后的R矩阵了。（相当于一个两层的神经网络）

3.1.4 LAYER NORMALIZATION和残差连接

们在上一步得到了经过注意力矩阵加权之后的 V, 也就是
tention(Q, K, V),
它进行一下转置, 使其和Xembedding的维度一致, 也就是
batch size, sequence length, embedding dimen
后把他们加起来做残差连接, 直接进行元素相加, 因为他
度一致:
mbedding+Attention(Q, K, V)
之后的运算里, 每经过一个模块的运算, 都要把运算之前
运算之后的值相加, 从而得到残差连接,
练的时候可以使梯度直接走捷径反传到最初始层:
SubLayer(X)


一个Transformer编码块完成
Xhidden= R = Norm(R + X_Attention))
Xhidden [4, 512]
R = Activate(Linear(Linear(Z))
R [4, 512]
Z = Norm(Xembeding_pos + Attention(Q,K,V))
Z [4, 512]
这个地方看一下Norm的过程:
X = Xembeding_pos + X_attention [4, 512]
Norm操作:
首先求每一行的均值ui
然后求每一行的方差σi的平方
LayerNorm(X) = α (Xij - ui) / √(σi²+ξ) + β
Attention(Q,K,V) = softmax(QK^T/√dk)V
Attention(Q,K,V) [4, 512]
dk表示head的个数，这里是8
V的每一行表示每个字向量的数学表达，而Q*K的转置为每个字与其他字的相关程度
head1 head2 head 8
head1 head2 head8
head1 head2 head 8
Q
K
V
Linear WQ
Linear WK
Linear WV
WQ, WK, WV [512, 512]
Q, K, V [4, 512]
Q =Linear(Xembedding_pos)
= Xembeding_pos WQ
K =Linear(Xembedding_pos)
= Xembeding_pos WK
V =Linear(Xembedding_pos)
= Xembeding_pos WV
Why
do
we
work
Xpos + Xembeding = Xembeding_pos
Xembeding_pos [4, 512]
why_pos
do_pos
we_pos
work_pos
Positional Encoding
Xpos [4, 512]
why
do
we
work
Input Embedding
Xembedding [4, 512]
Embedding 维度是512 dmodel = 512
Why do we work?
X [1, 4]
max sequence length = 4



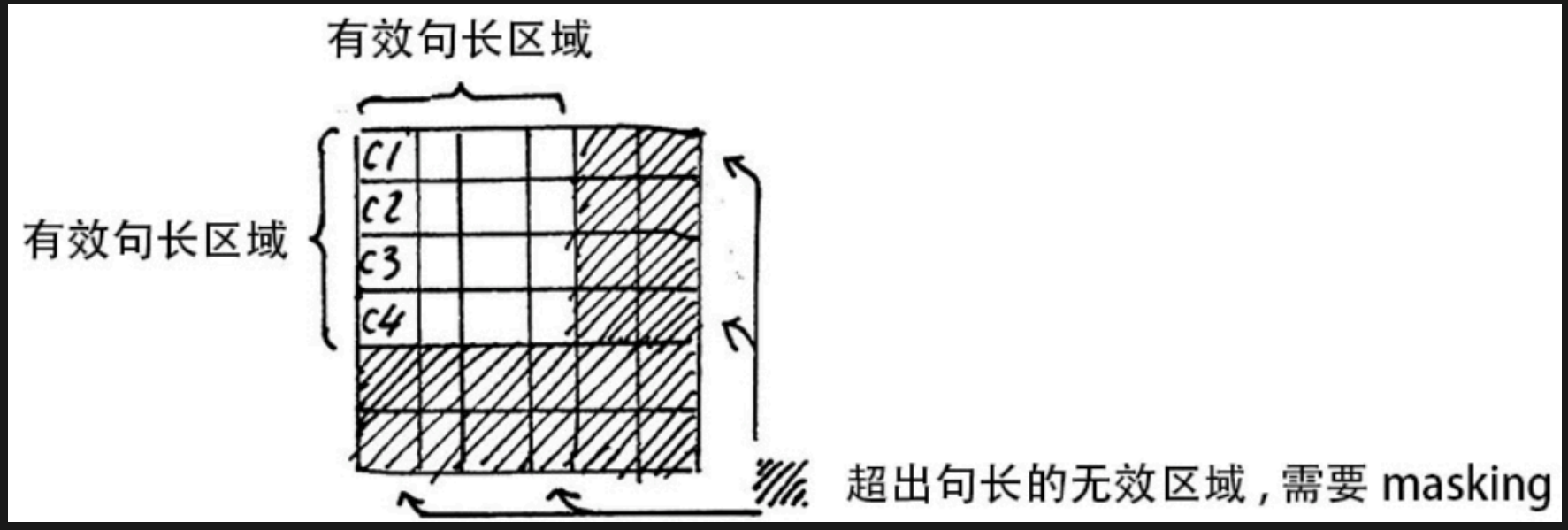
https://blog.csdn.net/wuzhongqiang

下面再说两个细节就可以把编码器的部分结束了

- 第一个细节就是：上面只是展示了一句话经过一个Transformer编码块之后的状态和维度，
- 但我们实际工作中，不会只有一句话和一个Transform编码块，所以对于输入来的维度一般是[batch_size, seq_len, embedding_dim]， 这里batch_size就是句子的数量（一个批次的样本数）
- 而Transformer编码块的个数一般也是多个，不过每一个的工作过程和上面一致，无非就是第一块的输出作为第二块的输入，然后再操作。
- 论文里面是用的6个Transformer块进行的堆叠。

PADDING MASK的问题：

- 如果有多句话的时候，句子都不一定一样长，
- 而我们的seq_len肯定是以最长的那个为标准，
- 不够长的句子一般用0来补充到最大长度，
- 这个过程叫做padding。



但这时在进行softmax的时候就会产生问题

- 回顾softmax函数

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, e^0$$

- 用0来补充的e^0是1, 是有值的,
- 这样的话 softmax中被 padding的部分就参与了运算,
- 就等于是让无效的部分参与了运算, 会产生很大隐患,

- 这时需要做一个mask让这些无效区域不参与运算,
- 我们一般给无效区域加一个很大的负数的偏置,

也就是:

$$z_{illegal} = z_{illegal} + bias_{illegal}$$

$$bias_{illegal} \to -\infty$$

$$e^{z_{illegal}} \to 0$$

- 经过上式的Padding masking我们使无效区域经过softmax计算之后还几乎为0, 这样就避免了无效区域参与计算.
- Transformer里面有两种mask方式
  - padding mask 和 sequence mask
  - 上面这个就是padding mask， 这种mask在scaled dot-product attention 里面都需要用到，
  - 而 sequence mask 只有在 Decoder 的 self-attention 里面用到。后面会看到。

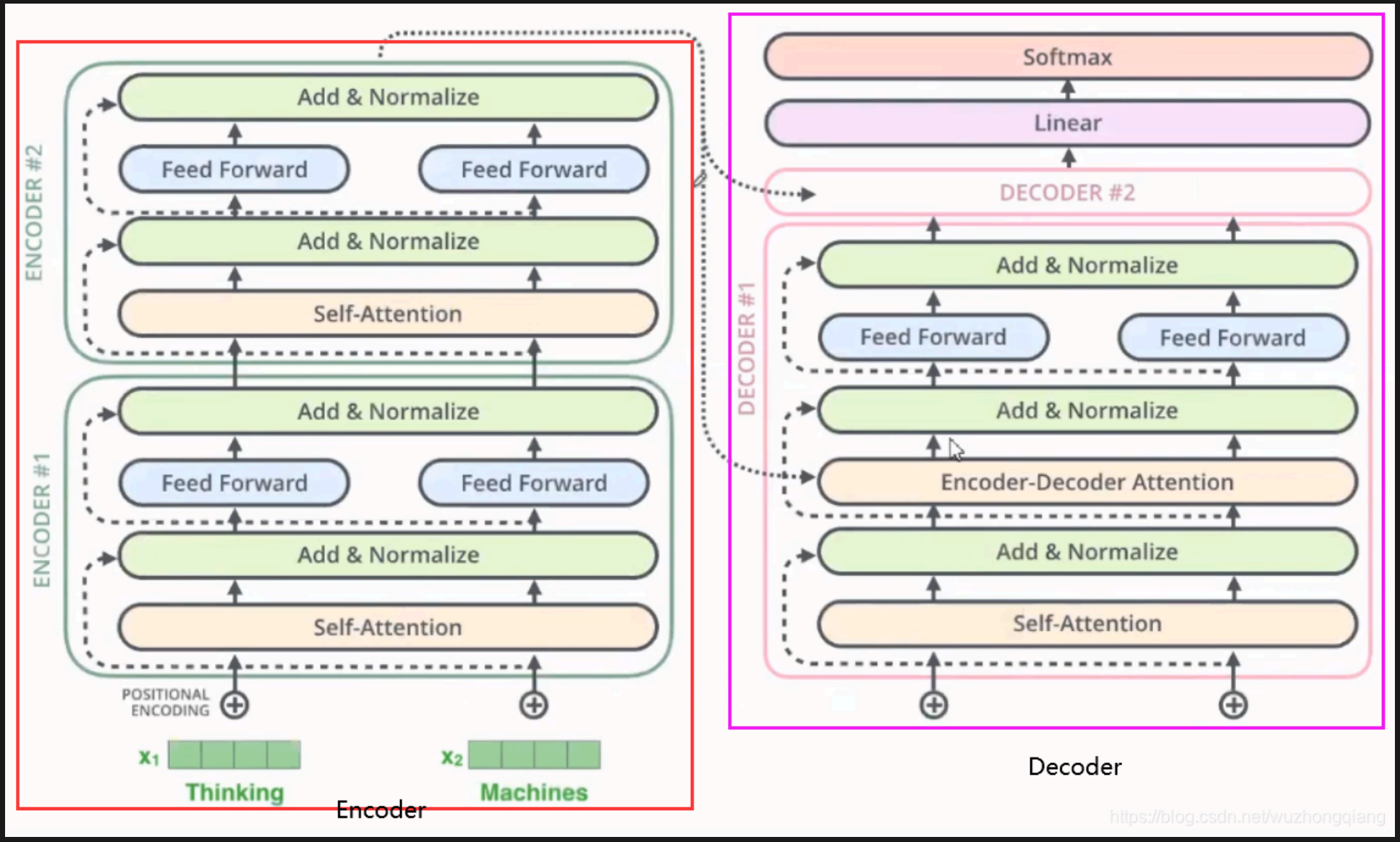
- 实际实现的时候， padding mask 实际上是一个张量，
- 每个值都是一个Boolean，值为 false 的地方就是我们要进行处理的地方(加负无穷的地方)
- 最后通过上面的梳理，我们解决了Transformer编码器部分，

- 总结：一个编码器Block = "注意力机制" + "前馈网络"。

    1. **输入** X
    2. X_attention = 多头注意力(X)
    3. X_1 = X + X_attention (残差连接)
    4. Z = LayerNorm(X_1) (归一化)
    5. Z_ffn = FeedForward(Z)
    6. Z_1 = Z + Z_ffn (残差连接)
    7. **输出** = LayerNorm(Z_1)

- 然后把这个"输出"再扔给下一个Block，重复6次

好了，编码器部分结束，下面进入解码器部分：



https://blog.csdn.net/wuzhongqiang

3.2 解码器部分的工作细节

解码器，看上面这张图

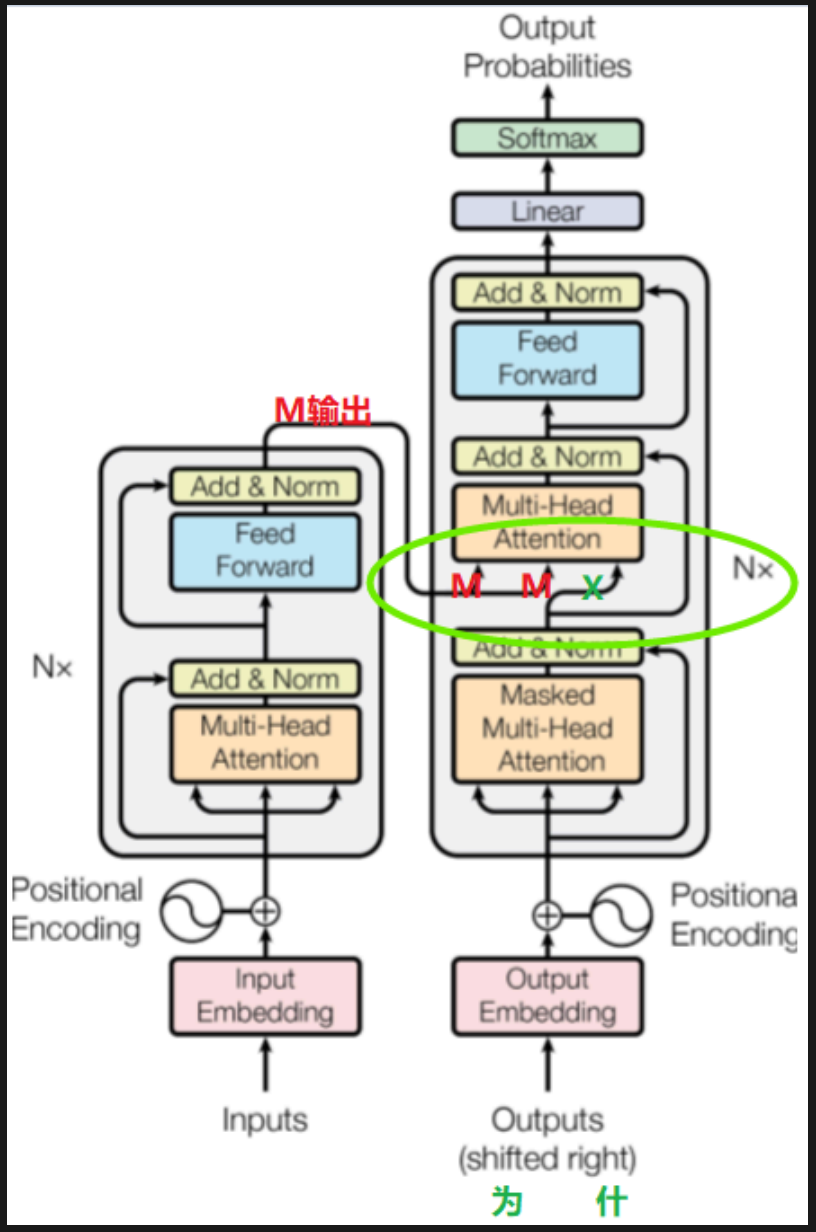
- 我们发现编码器和解码器其实差不多，
- 只不过解码器部分多了一个Encoder-Decoder Attention，
- 知道编码器是怎么工作的，也基本会解码器了，
- 但是还是来看几个细节

编码器通过处理输入序列开启工作

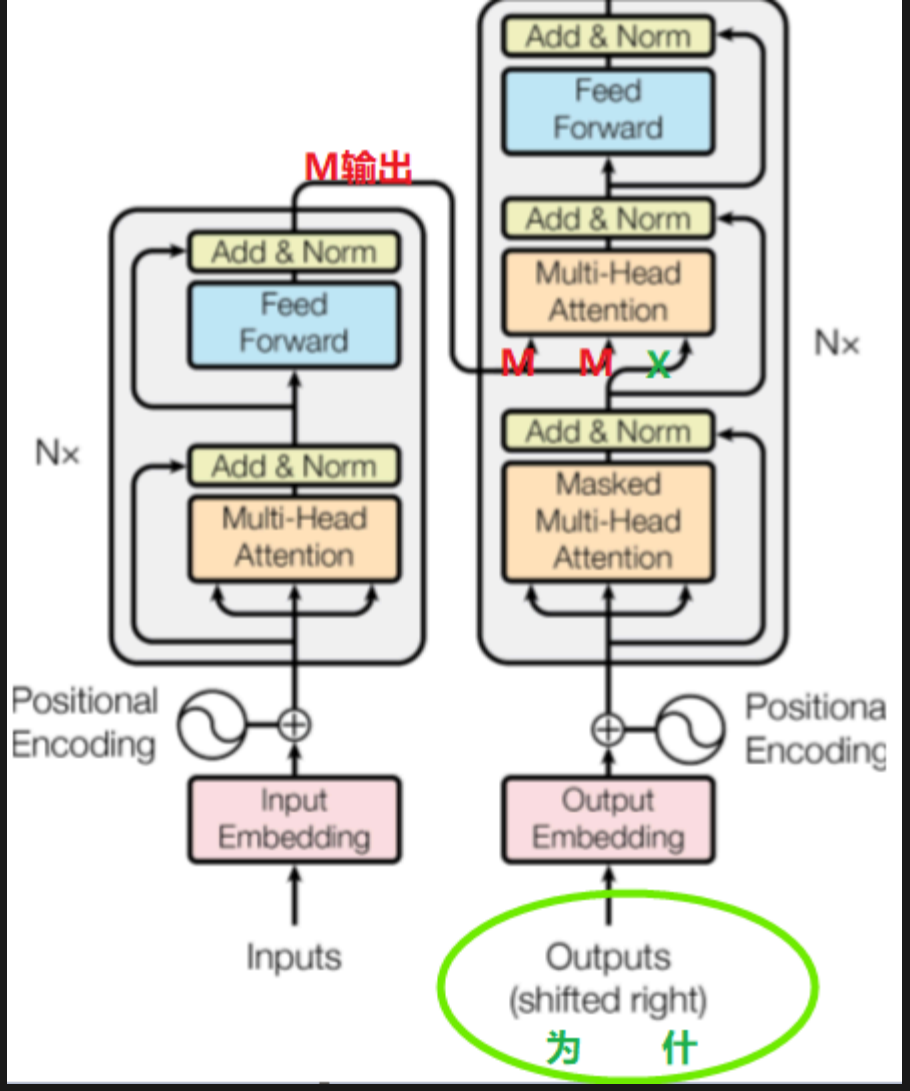
$$\text{Attention}(Q,K,V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Attention(Q,K,V) [4, 512]

- 顶端编码器的输出之后会变转化为一个**包含向量Q（查询向量）向量K（键向量）和V（值向量）的注意力向量集**
- 也就是**从多角度集自身与其他各个字关系的矩阵**，比如记为M。
- 这些向量将被每个解码器用于自身的“编码-解码注意力层”，
- 而这些层可以帮助解码器关注输入序列哪些位置合适。


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Nx
M输出
M M X
Positional Encoding
Positiona Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)
为 什

- 在完成编码阶段后，则开始解码阶段。
- 解码阶段的每个步骤都会输出一个输出序列（在这个例子里，是英语翻译的句子）的元素（先输出"为"，"为"落下去，输出"什"，"什"落下去输出"么"）
- 接下来的步骤重复了这个过程，直到到达一个特殊的终止符号，
- 它表示transformer的解码器已经完成了它的输出。



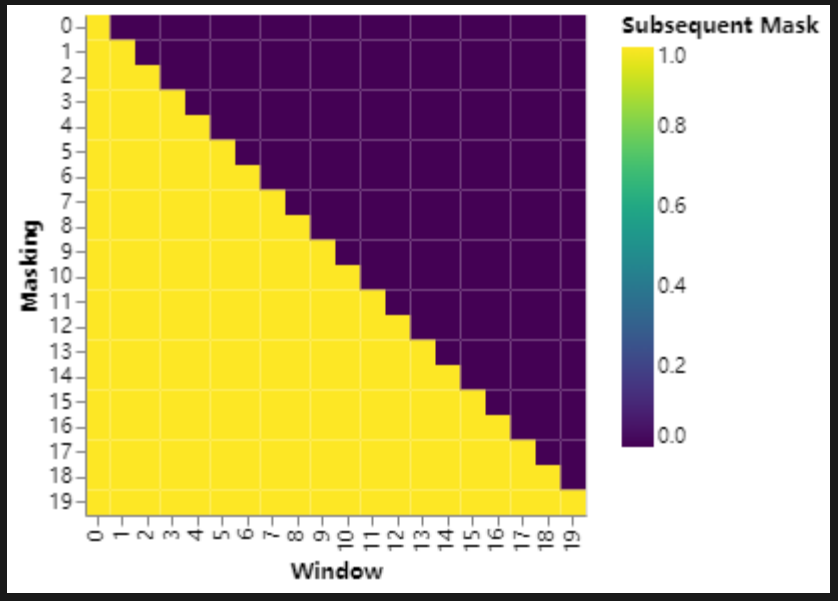
- 每个步骤的输出在下一个时间步被提供给底端解码器，并且就像编码器之前做的那样，这些解码器会输出它们的解码结果。

- 另外，就像我们对编码器的输入所做的那样，我们会嵌入并添加位置编码给那些解码器，来表示每个单词的位置。

解码器中的自注意力层表现的模式与编码器不同：

- 在解码器中，自注意力层只被允许处理输出序列中**更靠前的那些位置**。
- 在softmax步骤前，它会把后面的位置给隐去（把它们设为-inf），
- **这个mask操作的目的是不能让当前位置的词看到它之后的，只能看到它以及它之前的**，称为sequence mask。
- 这个具体实现的时候:

- 拿到注意力矩阵， 用一个上三角矩阵，上三角的值全为0



- 相当于把注意力矩阵的上三角部分变为0， 那么再乘V的时候，
- 当前的位置就无法看到后面的词信息了

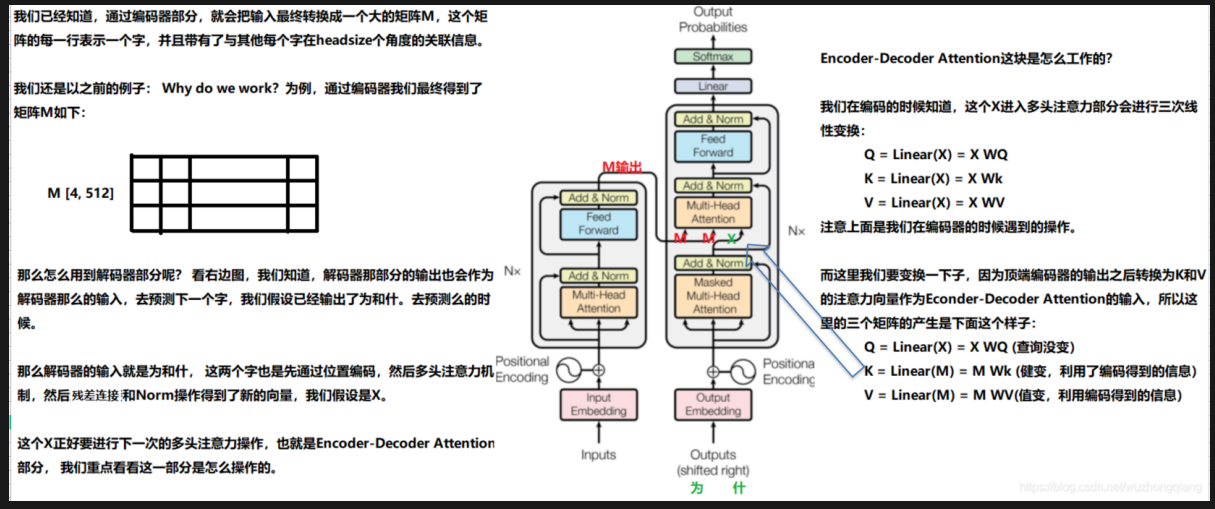
- 所以对于 decoder 的 self-attention，里面使用到的 scaled dot-product attention，同时需要 padding mask 和 sequence mask 作为 attn_mask，
- 具体实现就是两个mask相加作为attn_mask。

举个例子：

- 假设最大允许的序列长度为10，
- 先令padding mask为: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]，
- 然后再假设当前句子一共五个单词，
- 在输入第三个单词的时候， 前面有一个开始标识和两个单词， 那么此刻的sequence mask为[1, 1, 1, 0, 0]，
- 然后两个mask相加，得[1, 1, 1, 0, 0, 0, 0, 0, 0, 0]。

- 那么这个sequence mask到底是为什么要做呢?
- 比如 我/爱/参观/博物馆. 在输出爱的时候，如果不mask，会用到后面参观博物馆的信息。
- But， 我们做预测的时候， 预测当前输出，是看不到后面句子的， 毕竟后面句子还没有出来
- 所以如果**训练的时候不mask，就会导致训练和预测出现了一个gap**，所以这个sequence mask是必须要的。

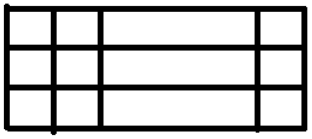
- 这个“编码-解码注意力层”工作方式基本就像多头自注意力层一样，
- 只不过**它是通过在它下面的层来创造查询矩阵，并且从编码器的输出中取得键/值矩阵。**
- 这个地方简单说一下细节（图片放大到下一页）



我们已经知道，通过编码器部分，就会把输入最终转换成一个大的矩阵M，这个矩阵的每一行表示一个字，并且带有了与其他每个字在headsize个角度的关联信息。

我们还是以之前的例子： Why do we work? 为例，通过编码器我们最终得到了矩阵M如下：

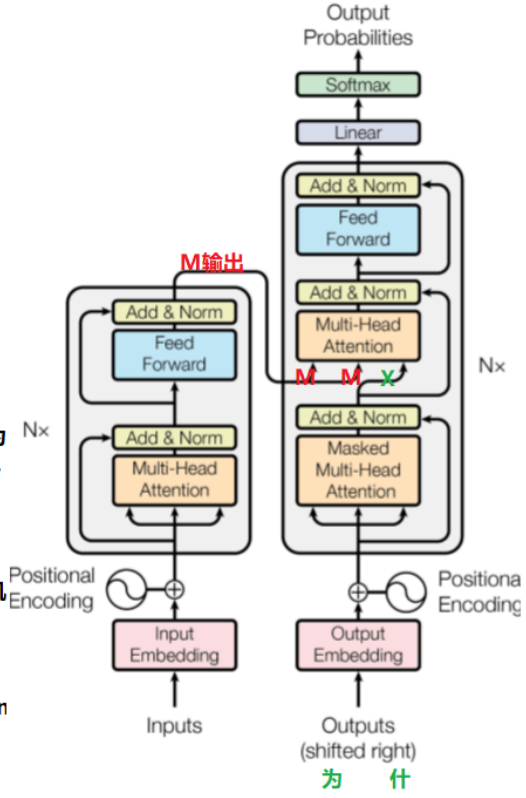
M [4, 512]



那么怎么用到解码器部分呢？ 看右边图，我们知道，解码器那部分的输出也会作为解码器那么的输入，去预测下一个字，我们假设已经输出了为和什。去预测么的时候。

那么解码器的输入就是为和什， 这两个字也是先通过位置编码，然后多头注意力机制，然后残差连接和Norm操作得到了新的向量，我们假设是X。

这个X正好要进行下一次的多头注意力操作，也就是Encoder-Decoder Attention部分， 我们重点看看这一部分是怎么操作的。



Encoder-Decoder Attention这块是怎么工作的?

我们在编码的时候知道，这个X进入多头注意力部分会进行三次线性变换：

Q = Linear(X) = X WQ

K = Linear(X) = X Wk

V = Linear(X) = X WV

注意上面是我们在编码器的时候遇到的操作。

而这里我们要变换一下子，因为顶端编码器的输出之后转换为K和V的注意力向量作为Econder-Decoder Attention的输入，所以这里的三个矩阵的产生是下面这个样子：

Q = Linear(X) = X WQ (查询没变)

K = Linear(M) = M Wk (健变，利用了编码得到的信息)

V = Linear(M) = M WV(值变，利用编码得到的信息)

https://blog.csdn.net/wuzhongqiang

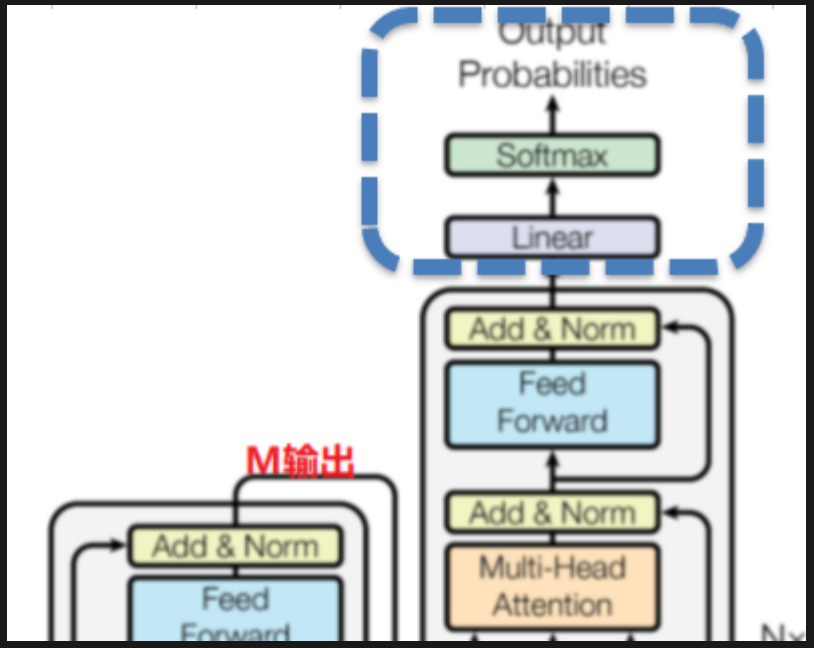
提问：在Transformer解码器中，sequence mask的作用是什么？

- A. 允许解码器看到所有输入信息
- B. 防止解码器看到未来的信息
- C. 加速训练过程
- D. 减少内存消耗

参考答案：B

选项分析：

- A: sequence mask限制解码器只能看到过去的信息，而不是所有输入信息。
- B: 正确答案，sequence mask用于防止解码器看到未来的信息。
- C: sequence mask与训练速度无直接关系。
- D: sequence mask与内存消耗无直接关系。



- 解码组件最后会输出一个实数向量。
- 我们如何把浮点数变成一个单词？这便是线性变换层要做的工作，它之后就是Softmax层。
- 线性变换层是一个简单的全连接神经网络，
- 它可以把解码组件产生的向量投射到一个比它大得多的、被称作对数几率（logits）的向量

里。

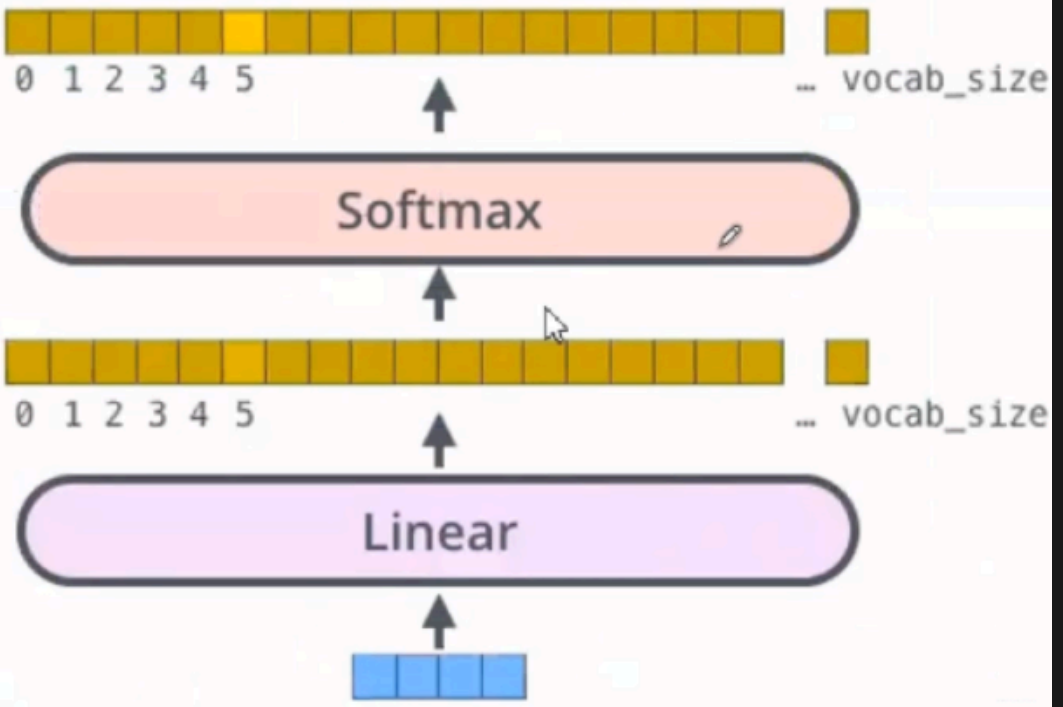
- 不妨假设我们的模型从训练集中学习一万个不同的英语单词（我们模型的“输出词表”）。
- 因此对数几率向量为一万个单元格长度的向量——每个单元格对应某一个单词的分数。
- 接下来的Softmax 层便会把那些分数变成概率（都为正数、上限1.0）。
- 概率最高的单元格被选中，并且它对应的单词被作为这个时间步的输出。

Which word in our vocabulary is associated with this index?

am

Get the index of the cell with the highest value (argmax)

5



Decoder stack output

这张图片从底部以解码器组件产生的输出向量开始。之后它会转化出一个输出单词。

4. TRAINING中的损失函数

我们已经过了一遍完整的transformer的前向传播过程，

- 下面可以直观感受一下它的训练过程。
- 在训练过程中，一个未经训练的模型会通过一个完全一样的前向传播。
- 但因为我们用有标记的训练集来训练它，所以我们可以用它的输出去与真实的输出做比较。

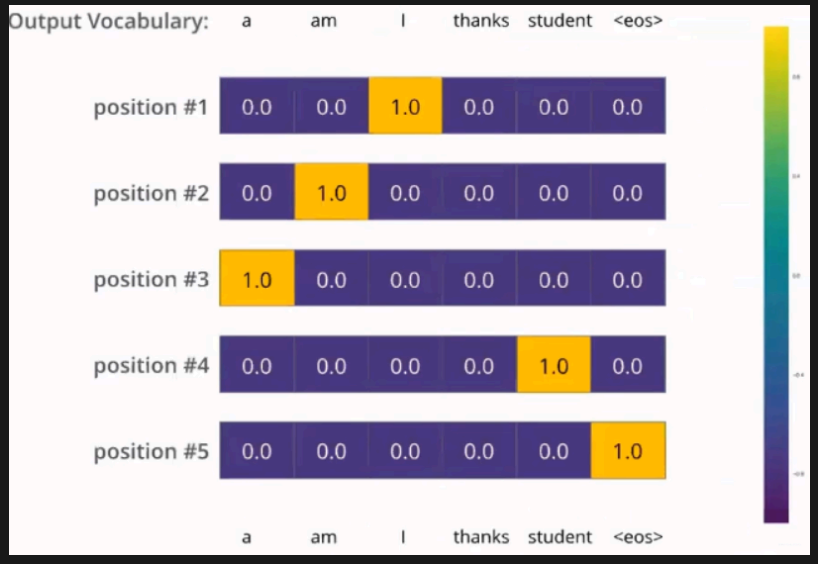
- 为了把这个流程可视化，不妨假设我们的输出词汇仅仅包含六个单词：“a”, “am”, “i”, “thanks”, “student”以及 <eos>（end of sentence的缩写形式）。

Output Vocabulary

WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

- 我们模型的输出词表在我们训练之前的预处理流程中就被设定好。

- 一旦我们定义了我们的输出词表，我们可以使用一个相同宽度的向量来表示我们词汇表中的每一个单词。
- 这也被认为是一个one-hot 编码。
- 所以，我们可以用下面这个向量来表示单



词“am”：

4.1 损失函数

那么我们的损失函数是什么呢?

- 这里使用的是交叉熵损失函数,
- 因为模型的参数（权重）都被随机的生成，（未经训练的）模型产生的概率分布在每个单元格/单词里都赋予了随机的数值。

- 我们可以用真实的输出来比较它，
- 然后用反向传播算法来略微调整所有模型的权重，生成更接近结果的输出。



- 左边就是我们想要的模型输出，
- 右边是我们训练的模型的输出，是一些概率的形式，
- 训练的时候，我们就先采用前向传播得到一个输出，
- 然后采用交叉熵损失比较模型的输出和真实的期望值，得到梯度反向传播回去更新参数。

4.2 训练小技巧

1. LabelSmoothing(regularization)

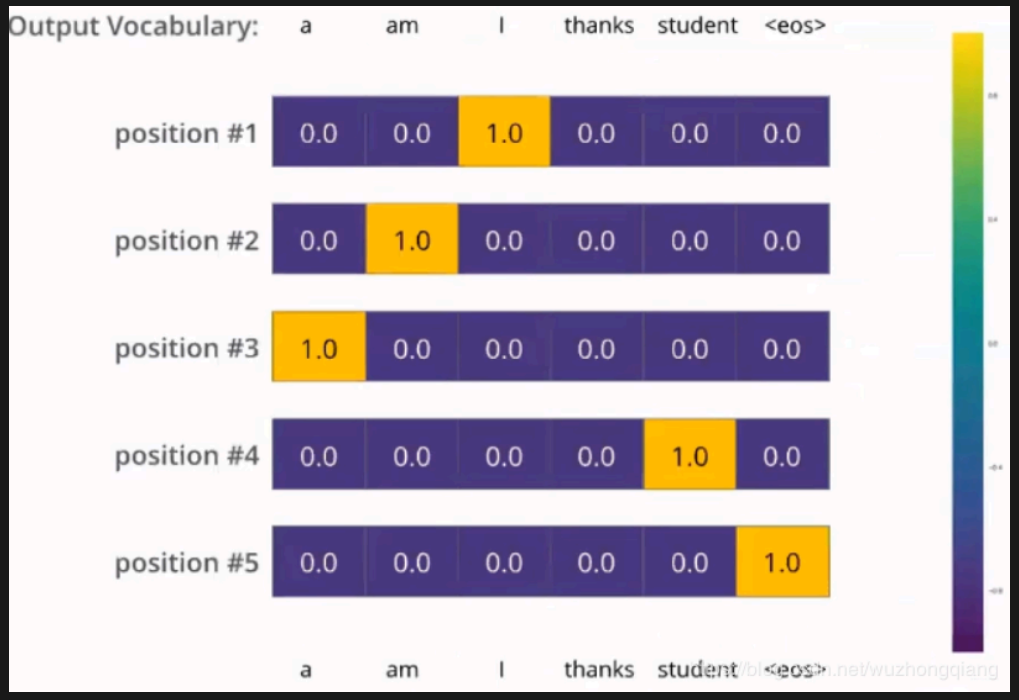
$$P_i = \begin{cases} 1, if\ (i = y) \\ 0, if\ (i \neq y) \end{cases} \Longrightarrow P_i = \begin{cases} (1-\varepsilon), if\ (i = y) \\ \dfrac{\varepsilon}{K-1}, if\ (i \neq y) \end{cases}$$

Note：K表示多分类的类别总数

ε是一个较小的超参数

- **问题：** 总是告诉模型"正确答案100%是'工'，其他0%"（one-hot编码），这太绝对了。这会让模型变得"死板"和"过度自信"。
- **技巧：Label Smoothing (标签平滑)**
    - 我们告诉它，"工"的正确概率是 **90%** (比如 `0.9`)。
    - 剩下的 **10%**，我们_平均分配_给字典里所有其他字（比如每个字0.0001%）。
- **好处：** 这让模型在训练时"保留一丝不确定性"，不会过于"死记硬背"，能提高模型的"泛化能力"（在没见过的数据上表现更好）。

比如上面我们的答案



我们最好不要标成这种形式，而是比如position #1这个，我们虽然想让机器输出I

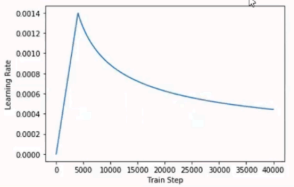
我们可以I对应的位置是0.9， 剩下的0.1其他五个地方平分，也就是

0.02　　0.02　　0.9　　0.02　　0.02　　0.02

2. Noam Learning Rate Schedule

这是一种非常重要的方式，如果不用这种学习率的话，可能训练不出一个好的Transformer。

$$lrate = d_{model}^{-0.5} * min(step_num^{-0.5}, step_num * warmup_steps^{-1.5})$$



- 简单的说，就是先让学习率线性增长到某个最大的值，然后再按指数的方式衰减。
- 相当于先有一个热启动阶段
  - 在这个阶段先让学习率线性增大，
  - 到一定的程度， 再开启冷启阶段，或者叫学习率衰减的方式。
  - 这个方式对于训练Transformer来说，是一个比较重要的策略。

5. CONCLUSION

这篇文章最经典的核心就是transformer结构

- 这种结构完全依赖于注意力机制，取代了基于Encoder-Decoder的循环层，
- 并且引入了位置嵌入，
- Multi-Head Attention机制。

- Transformer的核心是"自注意力机制"，它完全抛弃了RNN。
- 它通过"位置编码"来理解顺序。
- 它通过"多头注意力"从多角度理解句子。
- **优点：** 并行计算（快）、解决长距离依赖（不遗忘）。
- **缺点：** 理论上不是图灵完备的（RNN是）；对位置信息的处理方式有时比较"笨拙"。
- **但最重要的是：** 它的设计（尤其是Attention）为后续的BERT、GPT等模型铺平了道路，彻底改变了NLP领域。

代码实现：

- https://nlp.seas.harvard.edu/2018/04/03/attention.htr
- https://www.tensorflow.org/tutorials/text/transformer

参考：

- BERT大火却不懂Transformer？读这一篇就够了
- 从中文Transformer到BERT的模型精讲,以及基于BERT情感分类实战
- Self-Attention与Transformer